

广告投放规则与引擎机制

TeleHubX Campaign Dispatch Engine · 最后更新 2026-05-01

本文档定义 TeleHubX 广告投放 (Campaign) 的完整规则、调度算法与执行细节。任何修改 dispatch 代码必须先看这里，否则会破坏既定行为。

一、目标与发送号

1.1 目标解析顺序

广告投放的目标人群按以下顺序合并并去重：

- ☐ 客户群 (CampaignWizard Step 1 选的 customerGroupIds)
- ☐ 手动号码 (CampaignWizard Step 1 文本框填的 targets)
- ☐ 候选池 (如果客户群是 sourceType=candidates, 已包含候选池数据)

去重规则：电话规范化后比较；@username 小写比较；tgUserId 字符串比较。

1.2 发送号筛选 (智能模式)

条件	通过	拒绝
角色	ad / hybrid	cs
状态	online / offline	banned
健康度	healthScore $\geq$ 60	< 60 (黄/红)
隔离期	quarantineUntil 已过	还在隔离
接管状态	不在接管	在 HUMAN takeover
养号天数	$\geq$ 14 天	< 14 天 [软提示, 不阻止]

软提示规则：账号没养满 14 天时，启动会弹「承载不足」确认弹窗告知风险，但允许租户强制启动（测试场景必备）。

1.3 自定义槽位模式

租户可手动选号 (accountSourceMode>manual + adAccountIds[])。系统不再自动筛选，但仍跳过：接管中、隔离期、banned。

二、节奏档位

定义在 apps/server/src/campaigns/campaign.entity.ts 的 PACE_LIMITS:

档位	每号每天上限	时段数	适用场景
保守 (conservative)	20 条	3 时段	新号 / 风险敏感
平衡 (balanced)	30 条	3 时段	推荐默认
投放 (aggressive)	40 条	2 时段	成熟号 / 紧急活动

硬上限：无论档位，每号每天永远不能超过 40 条（TG 私聊安全线）。

三、目标分配算法

3.1 均分（默认）

目标数 $N = 300$
可用账号数 $K = 10$
每号配额 $quota = \text{ceil}(N / K) = 30$

3.2 跨天分配

如果 $quota >$ 档位上限，自动延伸到多天：

档位上限 $dailyLimit = 20$ （保守）
 $quota = 30$
天数 $days = \text{ceil}(30 / 20) = 2$
D1：每号发 20 条
D2：每号发 10 条

3.3 同 IP 组互斥

状态：Phase 2 实现，当前先不做

如果多个账号绑同一住宅代理 IP（同 proxyId 或同 boundIp）：同组账号不能同分钟发；组内串行：A 发完 $\rightarrow$ 等 30s $\rightarrow$ B 发。

3.4 分配顺序

为防止“按序号”全发完同一批人导致同一目标被同一号反复打：用账号 id 的稳定 hash 顺序循环分目标。每号拿到的 30 个目标是伪随机交错的，不是连续 30 个。

四、时段打散

4.1 时段定义

档位	时段窗口（本地时间）
保守（3 时段）	09:30-11:30 / 14:00-16:30 / 18:00-20:30
平衡（3 时段）	09:30-11:30 / 14:00-16:30 / 18:00-20:30
投放（2 时段）	10:00-14:00 / 16:00-20:00

全局规则：任何时段都不能跨过 09:00-21:00 本地时间窗口；时段之间有静默期（模拟午休/晚餐）。

4.2 时段内分配

每号在每个时段内分到 N 条，间隔走高斯分布：

时段长度 = 120 分钟
 $N = 7$ 条
平均间隔 $\mu = 120 / 7 \approx 17$ 分钟
标准差 $\sigma = \mu \times 0.4 \approx 7$ 分钟
每条间隔 = $N(\mu, \sigma)$ ，clamp 到 [60秒, 60分钟]

4.3 任务时间生成

每次发送任务的 `scheduledAt` = 时段起点 + 累计 Gaussian 间隔。调度精度到秒，多任务在同一秒不会冲突。

五、文案与开场白

5.1 文案池来源

配置	实际数据
<code>adTemplateId</code>	<code>ad_templates.variants</code> 取所有变体 + 原文
<code>adTemplateIds</code>	多模板的 <code>variants</code> 合并池
<code>messageVariants</code>	向后兼容老字段，直接作为变体池
<code>greetingMode</code> + <code>greetingTemplateIds</code>	选中开场白的 <code>text</code> + <code>variants</code>
<code>greetingMode = none</code>	不加开场白

5.2 选取规则

- ☐ 开场白: `none` 跳过 / `fixed` 始终用第一条 / `random` 池中随机
- ☐ 广告: 从广告变体池随机抽
- ☐ 复用控制 (Phase 2): 同号同变体最多复用 3 次

5.3 消息组装

默认 (一条合并): [开场白] + 空行 + [广告正文], 单条消息发出。

Phase 2 可选: 分两条发 (先开场白 → 5-15 秒 → 正文), 拟真度更高但风险翻倍。

六、失败处理与状态机

6.1 错误码处理

错误码	处理
<code>USER_PRIVACY_RESTRICTED</code>	该目标 <code>dead</code> , 跳过, 不重试
<code>PEER_ID_INVALID</code>	该目标 <code>dead</code> , 可选择从客户群移除
<code>USERNAME_NOT_OCCUPIED</code>	该目标 <code>dead</code>
<code>FLOOD_WAIT</code>	该号暂停 N 秒, 未发任务延后
<code>PEER_FLOOD</code>	该号当天停发, 明天恢复
<code>USER_BLOCKED_BY_ADMIN</code>	该号被某用户 <code>block</code> , 跳过该目标
网络错误	重试 1 次, 再失败标 <code>failed</code>

6.2 任务状态流转

```
pending → running → succeeded
                → failed
                → canceled
```

6.3 Campaign 级别状态

```
draft → scheduled → running → completed
                                → paused (用户暂停)
```

完成判定：所有子任务 (succeeded + failed + canceled) = 总任务数。

七、进度回写

7.1 每次发送后

- ☐ agent 调用 POST /campaigns/:id/sent 通知后端 +1
- ☐ 后端 campaign.sentCount++
- ☐ 候选池来源的目标 → 额外标记 lead_candidates.status = contacted

7.2 接收回复后

bot-gateway 收到客户回复时：检查 chatId 是否在某 campaign 的 targets 中 → 是则 campaign.replyCount++。

7.3 完成检测

每次任务状态更新时：查 campaign 下未完成任务数 → 0 时 campaign.status=completed, completedAt=now()。

八、安全护栏 (Guardrails)

无论档位如何，以下规则永远生效：

- ☐ 夜间保护：09:00-21:00 之外不发送
- ☐ 接管保护：发送前检查目标是否被人工接管，是则跳过
- ☐ 单号每日硬上限：40 条（即使档位是「投放」也不会超）
- ☐ 同号最小间隔：≥ 60 秒（防止极端异常）
- ☐ 租户暂停：tenant.status=suspended → 所有 campaigns 暂停
- ☐ License 失效：tenant.licenseExpiresAt < now() → dispatch 拒绝

九、测试场景的友好性

为了让租户能在没养号 14 天的情况下测试整套流程：

- ☐ 14 天养号是软提示，不阻止启动
- ☐ 「承载不足」时按钮变为「强制启动（承载不足）」
- ☐ 点击弹确认窗口告知风险，确认后才发送
- ☐ 测试场景下账号即使健康度低，仍会尝试发送（具体错误由 agent 上报）

十、引擎实现位置

模块	文件 / 方法
调度入口	apps/server/src/campaigns/campaign-dispatch.service.ts
目标解析	同上 — resolveTargets()

账号筛选	同上 — <code>selectAccounts()</code>
时段计算	同上 — <code>distribute()</code> + <code>PACE_WINDOWS</code>
任务生成	同上 — <code>dispatch()</code> 末尾 <code>taskRepo.save()</code>
Agent 执行器	<code>apps/agent/src/tasks/executors.ts</code> — <code>campaignSingle</code>
进度回写	<code>apps/server/src/campaigns/campaigns.controller.ts</code> — <code>POST /:id/sent</code>

十一、未来扩展 (Phase 2+)

- ☐ 同 IP 组互斥
- ☐ 同号同变体复用上限 (3 次)
- ☐ 开场白 / 正文分两条发的可选模式
- ☐ 失败号码自动从客户群移除 (可配置)
- ☐ A/B 测试: 同 campaign 自动比较 2 个变体的回复率
- ☐ 实时调速: 检测到回复率高 → 加快; 检测到 block 多 → 减速
- ☐ AI 评分的 mature account (不只是 14 天硬阈值)